Experiment No. 6

Aim: Implementation of Exception Handling in Java.

1. Problem Statement: Develop a BankAccount class that implements core banking operations:
○ balanceEnquiry(): Displays the current account balance.
○ withdraw(): Deducts the specified amount from the account balance.
○ deposit(): Adds the specified amount to the account balance.
Implement user-defined exceptions:
a. LowBalanceException: Thrown when a withdrawal amount exceeds the available balance.
b. NegativeNumberException: Thrown when attempting to deposit or withdraw a negative amount.
Develop a Java application program that demonstrates these functionalities and properly
handles these exceptions using try-catch blocks

```
import java.util.Scanner;

class LowBalanceException extends Exception {
    public LowBalanceException(String message) {
        super(message);
    }
}

class NegativeNumberException extends Exception {
    public NegativeNumberException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public void balanceEnquiry() {
        System.out.println("Current balance: " + balance);
    }
```

```
    public void withdraw(double amount) throws LowBalanceException,
NegativeNumberException {
        if (amount < 0) {
            throw new NegativeNumberException("Cannot withdraw negative amount.");
        }
        if (amount > balance) {
            throw new LowBalanceException("Insufficient balance.");
        }
        balance -= amount;
        System.out.println("Withdrawal successful. New balance: " + balance);
    }

    public void deposit(double amount) throws NegativeNumberException {
        if (amount < 0) {
            throw new NegativeNumberException("Cannot deposit negative amount.");
        }
        balance += amount;
        System.out.println("Deposit successful. New balance: " + balance);
    }
}

public class BankApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount account = new BankAccount(1000);
        while (true) {
            System.out.println("\n1. Balance Enquiry\n2. Deposit\n3. Withdraw\n4. Exit");
            System.out.print("Choose an option: ");
            int choice = sc.nextInt();
            try {
                switch (choice) {
                    case 1:
                        account.balanceEnquiry();
                        break;
                    case 2:
                        System.out.print("Enter deposit amount: ");
                        double depositAmount = sc.nextDouble();
                        account.deposit(depositAmount);
                        break;
                    case 3:
                        System.out.print("Enter withdrawal amount: ");
                        double withdrawAmount = sc.nextDouble();
                        account.withdraw(withdrawAmount);
                        break;
                    case 4:
                        System.out.println("Exiting...");
```

```
                sc.close();
                return;
            default:
                System.out.println("Invalid option.");
        }
    } catch (LowBalanceException | NegativeNumberException e) {
        System.out.println("Error: " + e.getMessage());
    }
  }
 }
}
```

```
PS D:\Prasanna\exp6> javac BankApp.java
PS D:\Prasanna\exp6> java BankApp

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 1
Current balance: 1000.0

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 2
Enter deposit amount: 500
Deposit successful. New balance: 1500.0

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 3
Enter withdrawal amount: 200
Withdrawal successful. New balance: 1300.0

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 3
Enter withdrawal amount: 2000
Error: Insufficient balance.

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 2
Enter deposit amount: -100
Error: Cannot deposit negative amount.

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 3
```

```
1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: 3
Enter withdrawal amount: -50
Error: Cannot withdraw negative amount.

1. Balance Enquiry
2. Deposit
3. Withdraw
4. Exit
Choose an option: |
```

2. Write a Java program with a method that takes an integer as input. If the number is odd,
the method should throw a custom exception (OddNumberException). Handle this
exception in the main program

```
import java.util.Scanner;

class OddNumberException extends Exception {
   public OddNumberException(String message) {
      super(message);
   }
}

public class OddNumberCheck {
   public static void checkEven(int num) throws OddNumberException {
      if (num % 2 != 0) {
         throw new OddNumberException("The number " + num + " is odd.");
      }
      System.out.println("The number " + num + " is even.");
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter an integer: ");
      int number = sc.nextInt();
      try {
```

```
        checkEven(number);
    } catch (OddNumberException e) {
        System.out.println("Exception caught: " + e.getMessage());
    }
    sc.close();
  }
}
```

```
PS D:\Prasanna\exp6> javac OddNumberCheck.java
PS D:\Prasanna\exp6> java OddNumberCheck
Enter an integer: 10
The number 10 is even.
PS D:\Prasanna\exp6> javac OddNumberCheck.java
PS D:\Prasanna\exp6> java OddNumberCheck
Enter an integer: 73
Exception caught: The number 73 is odd.
PS D:\Prasanna\exp6>
```

3.Create a package ExceptionHandlingDemo containing classes Calculator and DivisionException. ○ The Calculator class should have a method divide(int a, int b) that performs division. ○ If b is zero, throw a custom exception DivisionException with an appropriate error message. ○ Handle the exception in the main program and display an error message instead of crashing.

```
ExceptionHandlingDemo/
│
├── Calculator.java
├── DivisionException.java
└── MainProgram.java
```

// Calculator.java

```
package ExceptionHandlingDemo;

public class Calculator {
  public int divide(int a, int b) throws DivisionException {
    if (b == 0) {
      throw new DivisionException("Division by zero is not allowed.");
    }
    return a / b;
  }
}
```

//DivisionException.java

```
package ExceptionHandlingDemo;

public class DivisionException extends Exception {
    public DivisionException(String message) {
        super(message);
    }
}
```

// MainProgram.java

```
package ExceptionHandlingDemo;

import java.util.Scanner;

public class MainProgram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();
        System.out.print("Enter numerator: ");
        int a = sc.nextInt();
        System.out.print("Enter denominator: ");
        int b = sc.nextInt();
        try {
            int result = calc.divide(a, b);
            System.out.println("Result: " + result);
        } catch (DivisionException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}
```

```
PS D:\Prasanna\exp6> javac ExceptionHandlingDemo/*.java
PS D:\Prasanna\exp6> java ExceptionHandlingDemo.MainProgram
Enter numerator: 10
Enter denominator: 2
Result: 5
PS D:\Prasanna\exp6> javac ExceptionHandlingDemo/*.java
PS D:\Prasanna\exp6> java ExceptionHandlingDemo.MainProgram
Enter numerator: 10
Enter denominator: 0
Error: Division by zero is not allowed.
PS D:\Prasanna\exp6>
```